A Layered Architecture for Multi-Utility IoT Monitoring: Design and Implementation of a Building-Scale Metering Platform

Baxrom R. Reyimberganov[1,*]

[1]Department of Software Engineering, Urgench State University, Urgench, 220100, Uzbekistan

[*]Corresponding author: Baxrom R. Reyimberganov, Email: bahromreyimberganov0311@gmail.com, ORCID: 0009-0005-8124-0042

Keywords: IoT architecture, utility metering, DLMS/COSEM, RS-485, edge-to-cloud systems, FastAPI

Abstract

Utility monitoring in multi-apartment residential buildings — electricity, water, gas, soil moisture, ambient sound, and heating-loop temperature — is typically implemented as a collection of disjoint, single-purpose systems. This paper describes the design and implementation of a unified platform that ingests heterogeneous utility readings from ESP32-based sensor nodes into a single backend and presents them through a common dashboard. The platform combines a building-local RS-485 sensor bus with a per-building WiFi bridge, an asynchronous FastAPI/PostgreSQL backend, and a React single-page application. We describe the system's layered architecture, its device-authentication and data model, the rationale behind consolidating an earlier LoRa-mesh design into a simpler wired bus, and the operational infrastructure that serves it in production. We report concrete implementation metrics and discuss known architectural limitations, including a 32-bit timestamp representation that will require remediation before the year 2038.

1 Introduction

Multi-utility monitoring — observing electricity, water, gas, and environmental parameters across many physical points in a building or estate — is a recurring requirement in property management, and one that is poorly served by consumer smart-home products, which are typically single-utility and cloud-locked to a vendor. Purpose-built commercial metering platforms exist, but are often costly per endpoint and closed to customization, which matters when the building housing stock includes legacy analog meters (pressure switches, mechanical-interface electricity meters) that must be retrofitted rather than replaced.

This paper describes a platform built to address that gap for a specific, concrete deployment context: multi-apartment buildings where (a) each building already has a primary electricity meter compliant with the DLMS/COSEM standard (meter families TE71/TE73) reachable over RS-485, (b) supplementary sensors (water pressure, gas pressure, soil moisture, sound level, heating-loop temperature) are distributed at various points within the same building, and (c) only one reliable WiFi/internet uplink is assumed *per building*, not per sensor.

The central architectural question this constraint raises is: how should N heterogeneous, physically distributed sensors within one building reach a cloud backend if only one of them — or one dedicated node — has network connectivity? We describe an evolution from an earlier design (a LoRa mesh

network requiring a dedicated radio gateway per cluster) to a simpler wired-bus topology, and justify that evolution in terms of both engineering cost and operational reliability.

We contribute: (1) a description of a three-tier IoT-to-cloud architecture (RS-485 sensor bus → WiFi bridge → HTTP/WebSocket backend) suited to single-building, single-uplink deployments; (2) a device/data model that accommodates a passive-forwarding pattern, where a single network-visible "device" (the bridge) transparently carries readings from multiple physically distinct sensors while preserving per-sensor identity for traceability; (3) a report of the platform's current implementation scale and a discussion of the architectural debts accepted along the way.

2 System Design and Methodology

2.1 Overall topology

The platform is organized in four tiers:

1. **Sensor layer** — individual ESP32 microcontrollers ("leaf" nodes), each running a single-purpose firmware image for one utility type. A leaf node has no network stack of its own; it communicates exclusively over a two-wire RS-485 differential bus (A/B) shared by all leaves in the same building.

2. **Bridge layer** — one ESP32 per building, running a firmware image that (a) polls the RS-485 bus as bus master, collecting readings from every registered leaf, and (b) holds the building's only WiFi/HTTP uplink, relaying every collected reading to the backend over HTTPS.

3. **Backend layer** — a single, multi-tenant FastAPI service backed by PostgreSQL, exposing a REST API for device ingestion and a WebSocket channel for live dashboard updates.

4. **Presentation layer** — a React single-page application serving building operators, and, separately, a simplified kiosk-style "public display" endpoint for building-lobby screens.

A fifth, offline component — a PyQt6 desktop application — supports field technicians performing direct RS-485/DLMS diagnostics against a meter and flashing firmware onto new nodes, independent of the production network.

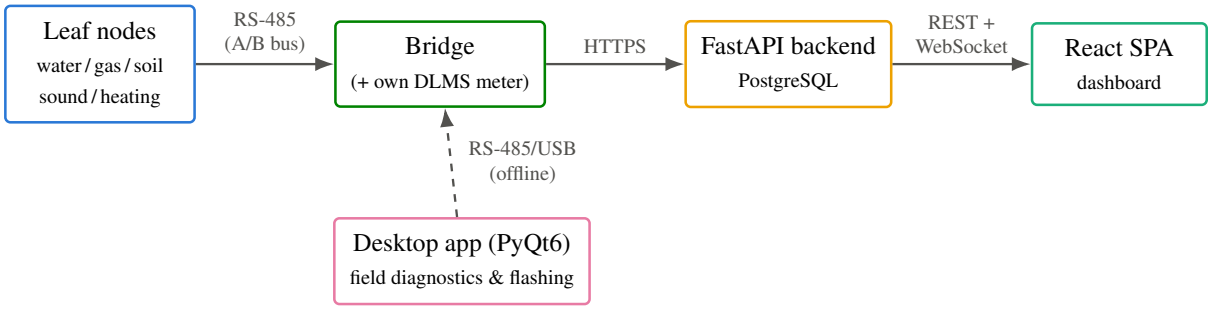

Figure 1: Four-tier architecture: RS-485 sensor leaves report to a per-building bridge, which relays over HTTPS to the backend; the frontend consumes the backend's REST/WebSocket API. A separate offline desktop tool (dashed edge) supports field diagnostics and firmware flashing, independent of the production data path.

2.2 RS-485 bus protocol

Communication on the building-internal bus follows a master–poll pattern to avoid collisions on the shared differential pair: the bridge periodically broadcasts a `DISCOVER` command; every leaf not already

registered in the bridge's roster responds, with a randomized jitter window to reduce the probability of two leaves replying simultaneously. Once registered, a leaf is addressed individually via a `POLL <id>` command and is expected to respond with a length-prefixed JSON payload describing its most recent reading.

Discovery is not a one-time boot-time event: the bridge re-runs `DISCOVER` on every polling cycle for the lifetime of the deployment, so a leaf that is slow to power up, or that is physically added to an already-running bus, is absorbed into the roster within one polling cycle rather than requiring a bridge restart. A leaf that has been addressed recently suppresses its own `DISCOVER` response for a bounded window T_{skip}, which prevents an already-registered, healthy leaf from re-announcing itself and colliding with a genuinely new leaf's discovery attempt.

For a roster of n leaves, each polled with a per-leaf reply window T_{reply} and up to r retries on non-response, the worst-case duration of one full polling cycle (all leaves unresponsive) is bounded by

$$T_{\text{cycle}}^{\max} \;=\; T_{\text{discover}} \;+\; n\left(r\,T_{\text{reply}} + T_{\text{gap}}\right) \tag{1}$$

where T_{discover} is the (bounded) discovery-round duration and T_{gap} is a fixed inter-leaf settling delay. This bound is what determines whether a bus-wide fault (e.g., a wiring break taking down every leaf simultaneously) can be tolerated without starving other periodic tasks — WiFi maintenance, watchdog service — that must also run within the same control loop; § 4 of the companion reliability paper discusses a concrete case where an unbudgeted $T_{\text{cycle}}^{\max}$ was found to approach several seconds and required tightening.

2.3 Electricity metering via DLMS/COSEM

Leaves (and the bridge itself, where directly wired) responsible for electricity readings communicate with the physical meter using the DLMS/COSEM application layer over HDLC framing, on a second RS-485 channel kept electrically and temporally separate from the building-internal sensor bus. The connection sequence attempts, in order: (1) an authenticated association at 9600 baud using the meter's "Client 1" logical device with HLS5 GMAC authentication; (2) on failure, a retry at 4800 baud; (3) on further failure, an unauthenticated association against "Client 16" (public/limited object set); (4) in an explicit test/demo mode, a simulated reading. Standard OBIS object codes are used to read voltage, current, active power, frequency, energy accumulation, and power factor per phase.

2.4 Data model and the "bridge-as-device" pattern

The backend's device/reading model was deliberately kept simple: a `Device` row represents anything with its own network identity — in practice, every bridge, and every leaf when standalone-WiFi firmware is used instead of RS-485-leaf firmware. A `Reading` row represents one timestamped measurement and carries a `utility_type` discriminator over a wide, mostly-sparse column set, rather than one normalized table per utility type — a pragmatic trade-off that keeps ingestion and query code uniform across utility types at the cost of many `NULL` columns per row.

The bridge/leaf topology required one addition: when the bridge forwards a leaf's reading, it does so *under its own device identity* (so the backend, and the operator dashboard, see one network-visible device per building rather than one per sensor) while attaching the leaf's own hardware identifier as a separate `source_id` field, preserving per-sensor traceability without multiplying the number of first-class "devices" an operator must manage. A subsequent schema revision introduced a dedicated `Sensor` entity with a `(sensor_uid, utility_type)` uniqueness constraint and a nullable foreign key from `Reading`, to formalize this many-sensors-per-device relationship independently of the wide-table `Reading` design.

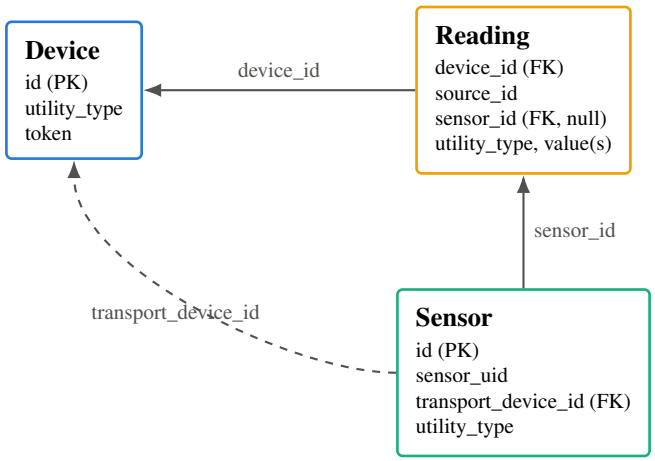

Figure 2: The bridge-as-device forwarding pattern: a bridge's own network identity is the `Device` row every `Reading` it forwards is attributed to (`device_id`); the leaf's own hardware identity is preserved separately as `source_id`. The later-introduced `Sensor` entity (dashed edge) formalizes the same many-sensors-per- bridge relationship as a nullable foreign key, additive to the original wide-table design.

2.5 Security model

Two independent authentication mechanisms operate at different trust boundaries. User-facing endpoints are protected by JWT bearer tokens with role-based access (`admin`/`user`/`viewer`); device-facing ingestion endpoints are protected by a per-device `X-Device-Token` header. An AI-assisted chat feature exposed to operators is deliberately prevented from issuing arbitrary database queries: it is restricted to a fixed, server-defined set of callable tools, with every privileged tool invocation independently re-checking the caller's role server-side rather than trusting the model's own judgment.

Several capabilities present in an earlier iteration were deliberately *removed* rather than merely deprecated: remote relay control (the ability to disconnect a meter over the network) was removed end-to-end following a decision that the capability's risk profile was not justified by its operational value; an earlier LoRa-mesh firmware generation's payload encryption was removed rather than retained, since its key was committed in the firmware source repository and therefore provided no real confidentiality guarantee.

2.6 Deployment and operational infrastructure

The backend runs as a single systemd-managed process on one modestly sized VPS instance, fronted by nginx, which also serves the frontend's static build and terminates TLS. Continuous integration is organized as three independent pipelines — backend, frontend, and firmware — each triggered only on changes within its own subtree; the backend and frontend pipelines additionally perform an automatic deploy-on-merge following a successful test/build run, while the firmware pipeline is compile-verification only. Over-the-air firmware updates are deliberately not implemented; updates are applied by direct USB flashing, a decision made after an earlier over-the-air mechanism was judged to add more operational risk than it removed.

3 Results

Table 1 summarizes the implementation's scale at the time of writing.

The consolidation from a LoRa-mesh architecture to the RS-485 bus/bridge architecture eliminated an entire class of hardware (dedicated radio gateway nodes) and an entire protocol layer (mesh routing, packet relay/flood, per-hop TTL handling) from the production deployment path; the LoRa firmware and

Component	Technology	Approx. size
Backend	Python 3.12, FastAPI, SQLAlchemy 2.0 (async), PostgreSQL, Alembic	∼15,000 lines; 30 migrations; 15 routers
Frontend	React 19, TypeScript, Vite, shadcn/ui, Tailwind CSS	∼87 source files
Firmware	C++17, PlatformIO / Arduino-ESP32	∼4,900 lines; 32 build configurations across 6 utility types and 3 deployment roles (standalone-WiFi, RS-485-leaf, bridge)
History	—	347 commits at time of writing

Table 1: Implementation scale across the three primary codebases.

its companion legacy frontend were archived to a separate branch rather than deleted outright, preserving the option to revisit the approach for a future deployment context without carrying that complexity in the actively maintained codebase.

The device/reading data model successfully accommodated the bridge-as-device forwarding pattern without requiring a schema change at the point the RS-485 architecture was introduced, validating the decision to keep `Reading` a wide, utility-type-discriminated table; the subsequent introduction of a first-class `Sensor` entity was additive (a nullable foreign key) rather than a breaking change, allowing the migration to proceed with an online, batched backfill against a live production table.

4 Discussion

Known architectural debt. All timestamp columns in the schema are stored as 32-bit `Integer` rather than a wider type — a decision inherited from an earlier development phase using SQLite (where the practical distinction is invisible) that only became a visible constraint after migrating to PostgreSQL, which enforces fixed-width integer storage. This is the classic Y2038 problem: every such column will overflow when

$$t_{\text{unix}} \;\geq\; 2^{31} - 1 \;=\; 2{,}147{,}483{,}647 \quad \text{seconds after the epoch,}$$

i.e., 03:14:07 UTC on 19 January 2038. The remediation — migrating to a 64-bit `BigInteger` representation across every timestamp column and every downstream consumer — is understood and deferred rather than unknown, but remains unresolved at the time of writing. We note this explicitly as a case study in how a storage-engine change can silently promote a previously invisible design limitation into an enforced one.

Consolidation debt. The migration from two parallel frontend codebases to a single actively maintained frontend, and from two firmware generations (LoRa-mesh and RS-485-bus) to one, both remain partially visible in the operational surface: a subset of pages exist in only one of the two frontend generations, a residual asymmetry from the migration proceeding page-by-page rather than atomically. We consider this an acceptable, explicitly tracked debt rather than an oversight, but flag it as a concrete example of the cost of incremental versus atomic architecture migrations in a system under continuous feature development.

Security posture. We note, as a limitation acknowledged but not yet remediated at the time of writing, that firmware images built in debug/development configurations bundle a single shared bootstrap device-authentication token and disable TLS certificate validation; this is an accepted trade-off for field-debugging convenience during the current deployment phase, but represents a fleet-wide impersonation risk if a debug-configuration binary or its source were to leak, and is recorded here as explicit unresolved work.

Threats to generalizability. The architecture described here is well suited to its specific deployment

assumption — one reliable WiFi uplink per building, with all sensors physically reachable by a low-cost twisted-pair bus from a single bridge point. It is a poor fit for deployments spanning distances or obstacles the RS-485 bus cannot practically cross, where the mesh-radio approach this system moved away from — or a hybrid retaining both approaches for different deployment contexts — would be the more appropriate choice; we regard the two as complementary rather than the wired-bus approach being a strict improvement in the general case.